

# Experiment - 7

**Aim: Built an ANN using backpropagation algorithms**

**Theory:**

## **ANN with Backpropagation**

1. Initialize: Define network (input, hidden, output layers), set small random weights and biases, choose activation function.
2. Forward Pass: Compute weighted sums for each neuron and apply activation to get outputs.
3. Compute Error: Compare network output with target using a loss function.
4. Backward Pass: Calculate deltas for output and hidden neurons using derivatives of activation (error gradients).
5. Update Weights/Biases : **Adjust weights:**  $W_{new} = W_{old} + \eta \cdot \delta \cdot input$
6. Iterate: Repeat forward and backward steps over epochs until error is minimized.
7. Test: Check performance on unseen data to ensure generalization.

**Code:**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix
import seaborn as sns

data = pd.read_csv("diabetes.csv")
X = data.iloc[:, :-1].values
y = data.iloc[:, -1].values.reshape(-1,1)
scaler = StandardScaler()
X = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

input_dim = X_train.shape[1]
hidden_dim = 10
output_dim = 1
lr = 0.05
```

```
epochs = 200
```

```
W1 = np.random.randn(input_dim, hidden_dim)
b1 = np.zeros((1, hidden_dim))
W2 = np.random.randn(hidden_dim, output_dim)
b2 = np.zeros((1, output_dim))
```

```
def relu(x):
    return np.maximum(0, x)
def relu_derivative(x):
    return (x > 0).astype(float)
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

```
loss_history = []
```

```
for epoch in range(epochs):
    Z1 = X_train.dot(W1) + b1
    A1 = relu(Z1)
    Z2 = A1.dot(W2) + b2
    y_pred = sigmoid(Z2)
```

```
    loss = np.mean((y_train - y_pred)**2)
    loss_history.append(loss)
```

```
    dZ2 = (y_pred - y_train) * (y_pred * (1 - y_pred))
    dW2 = A1.T.dot(dZ2) / X_train.shape[0]
    db2 = np.sum(dZ2, axis=0, keepdims=True) / X_train.shape[0]
    dA1 = dZ2.dot(W2.T)
    dZ1 = dA1 * relu_derivative(Z1)
    dW1 = X_train.T.dot(dZ1) / X_train.shape[0]
    db1 = np.sum(dZ1, axis=0, keepdims=True) / X_train.shape[0]
```

```
    W1 -= lr * dW1
    b1 -= lr * db1
    W2 -= lr * dW2
    b2 -= lr * db2
```

```
Z1_test = X_test.dot(W1) + b1
A1_test = relu(Z1_test)
Z2_test = A1_test.dot(W2) + b2
y_pred_test = sigmoid(Z2_test)
predictions = (y_pred_test > 0.5).astype(int)
```

```
cm = confusion_matrix(y_test, predictions)
```

```
plt.figure(figsize=(12,5))
```

```
plt.subplot(1,2,1)
```

```
plt.plot(range(epochs), loss_history)
```

```
plt.xlabel("Epochs")
```

```
plt.ylabel("Loss")
```

```
plt.title("Epoch vs Loss")
```

```
plt.subplot(1,2,2)
```

```
sns.heatmap(cm, annot=True, fmt='d')
```

```
plt.xlabel("Predicted")
```

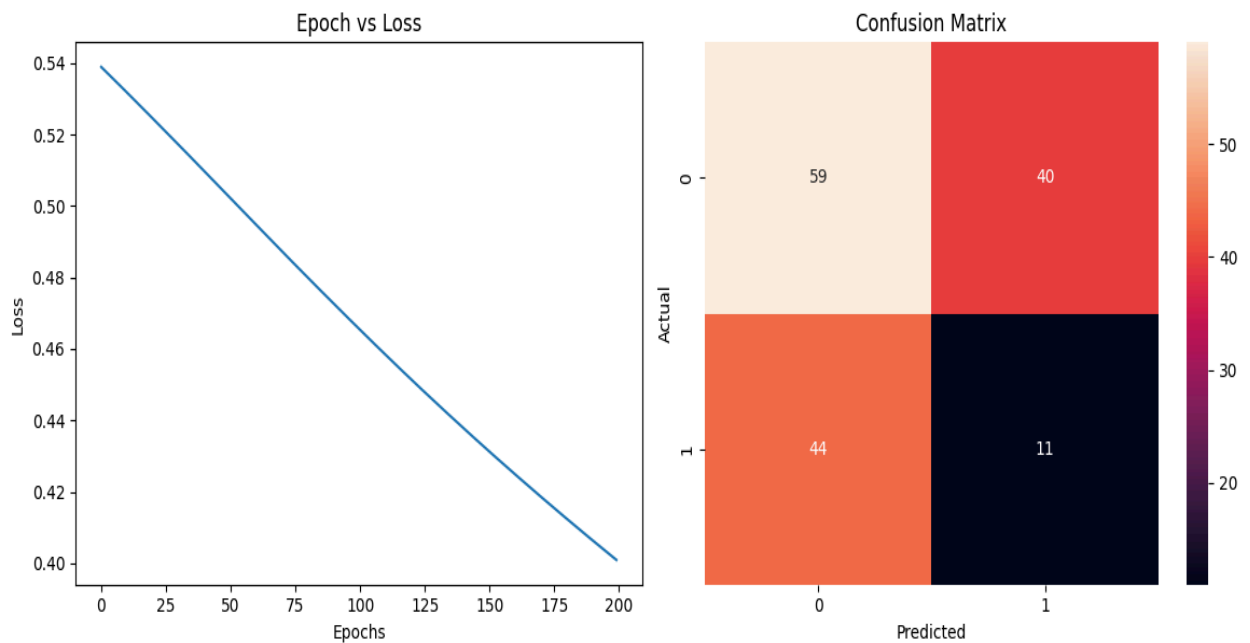
```
plt.ylabel("Actual")
```

```
plt.title("Confusion Matrix")
```

```
plt.tight_layout()
```

```
plt.show()
```

### Output:



## Experiment - 8

**Aim: To Implement LMS Algorithms for Adaline**

**Theory:**

**Adaline (Adaptive Linear Neuron)** is a single-layer neural network developed by Bernard Widrow and Ted Hoff. It uses a linear activation function and is trained using the LMS (Least Mean Squares) algorithm.

The LMS algorithm is used to minimize the error between the desired output and the predicted output by adjusting the weights iteratively.

The output of Adaline is given by:

$$y = \omega^T x + b$$

The error is calculated as:

$$e = d - y$$

The objective is to minimize the Mean Squared Error (MSE):

$$J = \frac{1}{2} e^2$$

The weights and bias are updated using the LMS rule:

$$w_{new} = w_{old} + \eta \cdot e \cdot x$$

$$b_{new} = b_{old} + \eta \cdot e$$

where:

- $\eta$  is the learning rate
- $e$  is the error
- $x$  is the input

This process is repeated for multiple epochs until the error is minimized.

**Code:**

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([[1],[2],[3],[4],[5]], dtype=float)
y = np.array([2,4,6,8,10], dtype=float)

X = (X - np.mean(X)) / np.std(X)

w = np.random.randn(1)
b = np.random.randn(1)

lr = 0.01
epochs = 50

losses = []

for epoch in range(epochs):
    y_pred = w * X.flatten() + b

    error = y - y_pred
    loss = np.mean(error**2)
    losses.append(loss)

    dw = -2 * np.mean(error * X.flatten())
    db = -2 * np.mean(error)

    w -= lr * dw
    b -= lr * db

    print(f"Epoch {epoch+1}, Loss: {loss:.4f}")

plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(range(epochs), losses)
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Epoch vs Loss")

plt.subplot(1,2,2)
plt.scatter(X, y, label="True")
plt.plot(X, w * X.flatten() + b, label="Prediction")
```

```
plt.xlabel("X")
plt.ylabel("y")
plt.title("Adaline Fit")
plt.legend()
```

```
plt.tight_layout()
plt.show()
```

## Output:

```
... Epoch 1, Loss: 41.5884
Epoch 2, Loss: 39.9415
Epoch 3, Loss: 38.3598
Epoch 4, Loss: 36.8408
Epoch 5, Loss: 35.3819
Epoch 6, Loss: 33.9808
Epoch 7, Loss: 32.6351
Epoch 8, Loss: 31.3428
Epoch 9, Loss: 30.1016
Epoch 10, Loss: 28.9096
Epoch 11, Loss: 27.7647
Epoch 12, Loss: 26.6653
Epoch 13, Loss: 25.6093
Epoch 14, Loss: 24.5952
Epoch 15, Loss: 23.6212
Epoch 16, Loss: 22.6858
Epoch 17, Loss: 21.7875
Epoch 18, Loss: 20.9247
Epoch 19, Loss: 20.0961
Epoch 20, Loss: 19.3003
```

